

Manuel utilisateur de autolisp-vector

OpenAI

April 27, 2026

Contents

1	Objet	1
2	Chargement	2
3	API publique	2
3.1	Création	2
3.2	Prédicats et introspection	3
3.3	Accès et mutation	4
3.4	Opérations de séquence	4
3.5	Fill-pointer et usage en pile	5
3.6	Affichage	5
4	Exemples complets	6
4.1	Vecteur simple	6
4.2	Initialisation depuis une liste	6
4.3	Pile bornée	6
5	Limitations de la v1	6
6	Tests	7

1 Objet

`autolisp-vector` fournit une abstraction de **vecteur** inspirée de Common Lisp pour AutoLISP.

Le module propose :

- création de vecteurs ;

- accès par indice ;
- mutation par indice ;
- sous-séquences ;
- copie ;
- opérations basées sur le `fill-pointer` ;
- conversion vers une liste ;
- affichage en syntaxe de vecteur Common Lisp.

La représentation interne est masquée derrière une API publique. En pratique, un vecteur est représenté par un symbole portant des propriétés internes.

2 Chargement

Exemple minimal :

```
(load "src/autolisp-vector.lsp")
```

3 API publique

3.1 Création

3.1.1 `(av-make-array size initial-element initial-contents fill-pointer)`

Crée un vecteur de longueur fixe.

Contraintes :

- `size` doit être un entier ≥ 0 ;
- `initial-element` et `initial-contents` sont mutuellement exclusifs ;
- si `fill-pointer` est fourni, il doit être dans l'intervalle $[0, \text{size}]$.

Cas d'usage typiques :

```
;; vecteur de 4 cases initialisées à 0
(setq v1 (av-make-array 4 0 nil nil))

;; vecteur initialisé depuis une liste
(setq v2 (av-make-array 4 nil '(10 20 30 40) nil))

;; vecteur de pile avec fill-pointer
(setq stack (av-make-array 8 nil nil 0))
```

3.1.2 (av-vector elements)

Construit un vecteur à partir d'une liste.

```
(setq v (av-vector '(a b c)))
```

3.2 Prédicats et introspection

- (av-vector-p obj)
- (av-vectorp obj)
- (av-arrayp obj)
- (av-length vec)
- (av-vector-length vec)
- (av-array-total-size vec)
- (av-adjustable-array-p vec)
- (av-fill-pointer vec)

Notes :

- la v1 n'implémente pas de vecteurs ajustables ;
- av-adjustable-array-p retourne donc toujours nil ;
- av-fill-pointer lit la valeur du fill-pointer courant.

3.3 Accès et mutation

- `(av-aref vec index)`
- `(av-svref vec index)`
- `(av-set-aref vec index value)`

Exemple :

```
(setq v (av-make-array 3 nil '(1 2 3) nil))  
(av-aref v 1)  
;; => 2
```

```
(av-set-aref v 1 99)  
(av-to-list v)  
;; => (1 99 3)
```

Les indices sont bornés. Un accès hors limites provoque une erreur.

3.4 Opérations de séquence

- `(av-replace destination source start1 end1 start2 end2)`
- `(av-subseq vec start end)`
- `(av-copy-vector vec)`
- `(av-to-list vec)`

Exemples :

```
(setq v (av-make-array 5 0 nil nil))  
(av-replace v '(7 8 9) 1 4 0 3)  
(av-to-list v)  
;; => (0 7 8 9 0)
```

```
(setq slice (av-subseq v 1 4))  
(av-to-list slice)  
;; => (7 8 9)
```

```
(setq copy (av-copy-vector v))
```

Le paramètre `source` de `av-replace` peut être :

- une liste ;
- un autre vecteur `autolisp-vector`.

3.5 Fill-pointer et usage en pile

- `(av-set-fill-pointer vec fill-pointer)`
- `(av-vector-push item vec)`
- `(av-vector-pop vec)`

Exemple :

```
(setq stack (av-make-array 3 nil nil 0))
```

```
(av-vector-push 'a stack)
;; => 0
(av-vector-push 'b stack)
;; => 1
```

```
(av-fill-pointer stack)
;; => 2
```

```
(av-vector-pop stack)
;; => B
```

Sémantique :

- `av-vector-push` retourne l'indice d'insertion ;
- si le vecteur est plein, `av-vector-push` retourne `nil` ;
- `av-vector-pop` exige un fill-pointer strictement positif.

3.6 Affichage

- `(av-vector-string vec)`
- `(av-print-vector vec)`

Exemple :

```
(setq v (av-make-array 4 nil '(1 2 3 4) nil))
(av-vector-string v)
;; => "#(1 2 3 4)"
```

Le format retenu suit la syntaxe usuelle des vecteurs Common Lisp.

4 Exemples complets

4.1 Vecteur simple

```
(setq v (av-make-array 4 0 nil nil))
(av-set-aref v 2 42)
(av-to-list v)
;; => (0 0 42 0)
```

4.2 Initialisation depuis une liste

```
(setq v (av-make-array 4 nil '(10 20 30 40) nil))
(av-subseq v 1 3)
;; => vecteur contenant 20 et 30
```

4.3 Pile bornée

```
(setq stack (av-make-array 2 nil nil 0))
(av-vector-push 'x stack)
(av-vector-push 'y stack)
(av-vector-push 'z stack)
;; => nil, car le vecteur est plein
```

5 Limitations de la v1

- pas de redimensionnement dynamique ;
- pas de `setf` compatible Common Lisp ;
- pas de tableaux multidimensionnels ;
- pas de distinction entre vecteur simple et vecteur général au sens complet de Common Lisp ;
- le stockage interne reste spécialisé pour les besoins du dépôt.

6 Tests

La suite de tests actuelle couvre notamment :

- création ;
- initialisation par `initial-contents` ;
- mutation ;
- `replace` ;
- `subseq` ;
- copie ;
- `fill-pointer` ;
- impression ;
- vérification des bornes.

Exécution :

```
make test-bricscad
```